实验三 手搓最小可执行文件

——ELF 文件与链接

作为 C 语言的初学者，第一个编写的 C 代码一般是如下所示的“Hello World”：

```
#include <stdio.h>
int main() {
    printf("Hello, World!");
    return 0;
}
```

但这区区四行 C 代码，在 X86 架构下 Ubuntu 中使用 gcc 形成的可执行文件大小可能超过 7KB ！

实验目标：

- 请尝试分析 TA 到底增加了什么内容导致可执行文件的大小产生这样的膨胀？
- 基于前述分析，如果一段极其简单的代码如下，TA 仅仅返回一个数字 XX（**请使用你学号的最后两位取代下面代码中的 “XX”**）：

```
int main ( ) { return XX; }
```

你可以尽你所能（后文我们会为你提供指导参考）获取功能没有任何变化的更小（字节数）可执行文件，你能获得多小的可执行文件？

实验三 Detail 指导

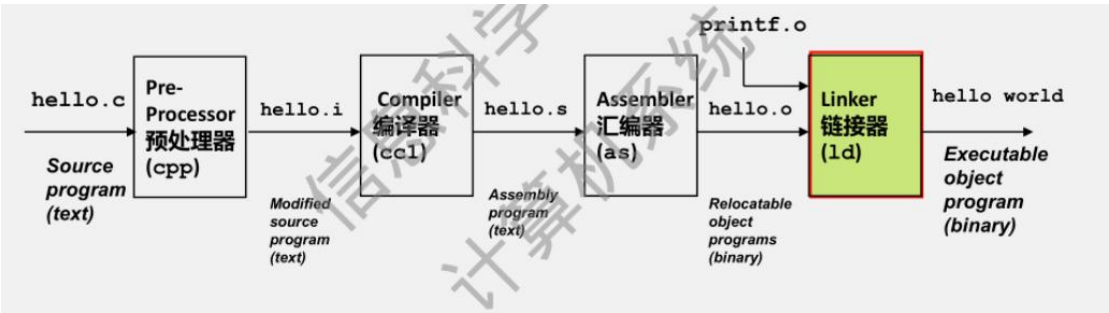
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~首先是友情建议~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

- 有鉴于 32 位系统和 64 位系统的差异性，本实验建议优选在 32 位系统中进行。
- 你可以使用命令 ./a.out ; echo $? 查看返回值以确定缩减后的可执行文件 a.out 的运行结果。
- 由于需要对汇编代码直接进行修改，请一定采用更有力的汇编语言编译器 nasm。
  - **安装**：可以通过包管理器安装，例如在 `Ubuntu` 上使用 `sudo apt install nasm`。
  - **编写代码**：使用文本编辑器编写汇编代码，文件扩展名通常为 `.asm`。
  - **编译代码**：使用命令 `nasm -f format filename.asm` 将汇编代码编译成目标文件。
  - **链接**：使用链接器（如 `ld`）将目标文件链接成可执行文件

~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~以下是正文~~~~~~

预备动作：

回顾一下源代码是如何优雅得变成可执行文件的：



那么，可以尝试的方法就只能在上述各个节点（包括编译、汇编、链接等）去进行“尽量小”尝试。接下来让我们一步步来考虑如何缩减可执行的大小。

Step 1:

- 前述仅仅返回你学号（最后两位）的代码，ta 的可执行文件有多大？
- 如果要减小可执行代码，其 C 源码层面还能进一步简化吗？如果不能，为什么？
- 求助于 gcc 编译器呢？ta 有优化选项啊？请尝试，是否缩小了可执行文件？

Step 2:

如果 Step1 中的步骤都尝试过了，优化尺寸了吗？下面我们尝试一下直接开整汇编代码。

先演练一下 nasm，以下是一个手写汇编的示例及其编译过程：

```
;Example.asm：
BITS 32          ；指定生成 32 位代码
GLOBAL main      ；声明 main 为全局符号
SECTION .text    ；定义代码段

main:
    mov eax, 5
    mov ebx, 10
    add eax, ebx
    ret
```

nasm 编译命令：

```
nasm -f elf32 Example.asm
gcc -m32 -Wall -s Example.o
```

Tip：你最好运行一下该示例并查看其返回值，这对你编写自己的汇编代码会有帮助。

Step 3:

- 本阶段，请使用 readelf 或 objdump 随时查看可执行 ELF 文件的格式和内容。
- 你是否注意到可执行 ELF 文件中包含与**链接有关**的部分。想一想，我们并没有明显调用外部库函数，为什么还会出现这些链接部分以及其有什么作用？
- 对于一个如此简单的程序，我们是否有必要去调用外部的库函数呢？
- 尝试使用 gcc 指令 -nostdlib 来取消链接标准库和启动代码。或许你在尝试后会遇到 _start 未定义的报错（这是由于取消链接后，无法调用外部函数引起的），而这又是程序运行的入口，所以只能由我们自己在汇编代码中进行定义，且由于此 C 程序所要执行的功能非常简单，所以我们完全可以让任务在 _start 中完成，这样就不需要再

定义 main 函数了。

- 如果你成功编译，在执行时可能会遇到**段错误**！？

  问题在于我们将 _start 视为 C 函数，并尝试从中返回。实际上，它根本不是一个函数。它只是目标文件中的一个符号，链接器使用它来查找程序的入口点。当我们的程序被调用时，它是直接调用的。查看一下，我们会看到堆栈顶部的值是数字 ，这看起来不像地址？事实上，堆栈上的是程序的 argc 值，在此之后是 argv 数组的元素，包括终止的 NULL 元素，然后是 envp 的元素——请回忆 argc，argv，envp 是干什么用的？所以此时堆栈上没有返回地址。

- 那么， **_start 如何退出**呢？与大多数作系统一样， Linux 通过系统调用为其托管的程序提供基本操作，包括打开文件、读取和写入文件句柄等，当然，还包括关闭进程。

  首先我们可以通过 int 0X80 这条指令唤醒内核—— 所以 int 0X80 有什么用？

  除了唤醒内核之外，还需要告诉内核我们想要**结束该进程**，以及该**进程返回值**。这需要对 eax 赋值为 1 （告诉内核结束进程），对 ebx 赋值为返回值——为什么只能对这两个寄存器进行赋值？

- 以上操作，将在可执行文件中剔除链接的相关部分，请查看其 ELF 文件结构和内容，与你的预期一致吗？

- 你可能注意到仍然还存在链接相关的内容，因为 gcc 在链接时会增添一些额外信息，但我们没有使用它的任何附加功能，因此我们可以自己调用**链接器 ld** ：

  ld -m elf_i386 Example.o //elf_i386 是指生成 32 位 x86 架构的 ELF 格式

Step 3.5：

在进行 Step 4 之前，我们需要对现在版本的汇编代码进一步缩减，不然在之后的步骤可能会遇到一点小问题（你可以尝试如果跳过这一步，观察后续会遇见什么问题，trytry）。当然你极有可能只能瞪着你桌面上看起来已经简单到发指的汇编代码发呆。别担心，我们会直接给出其缩减版，但是你需要去自己挖掘这样修改的依据

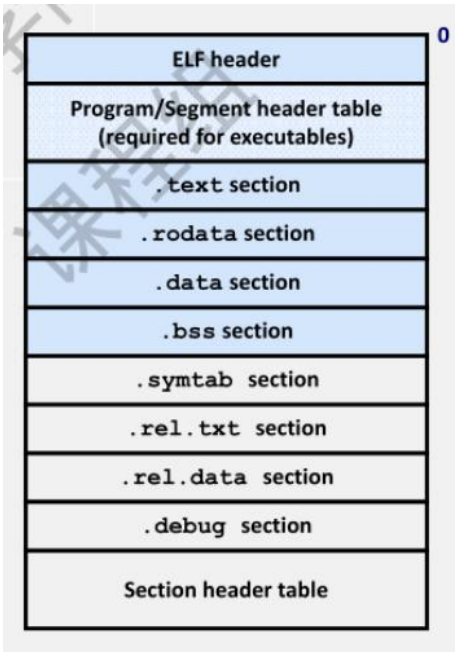
以下是_start 修改后的内容

```
xor eax, eax
inc eax
mov bl, XX
int 0x80
```

Step 4:

现在我们获得一个相对“简单”的可执行文件，ta 有多大？如果想进一步减小 ta，就只能直接从 ELF 文件下手了。

- 回忆一下 ELF 格式，课堂讲授大致如下图所示：



你看到的 ELF 文件格式与其一致吗？

- 每个 ELF 文件都以 ELF header 开头，其包含什么内容？
- Program header 里面是什么？
- .rodata ，.data ，.bss 等都各自有什么内容和作用？思考一下，对于我们极其简单的源程序来说，既没有需要调用的外部函数、也没有需要存储的变量，这些对于我们程序的实现有确实意义吗？
- 对于一个可执行文件来说，其必要的部分是 ELF header 、 Program header 以及 .text section ，至于其它部分，嗯……必需吗？
- 在锁定 ELF 文件中的必要部分后，我们可以开始尝试手搓一个更最简单的 ELF 文件（对，就像本课程最开始那两周一样，直接写汇编代码）。当然你可能还是对于如何写一个 ELF 文件毫无头绪，别担心，这里有一个标准的 ELF 文件，其只包含了上述所说的三个部分（ ELF header 、 Program header 、.text ）

```
BITS 32

            org     0x08048000

ehdr:                                           ; Elf32_Ehdr
            db      0x7F, "ELF", 1, 1, 1, 0     ;   e_ident
      times 8 db    0
            dw      2                           ;   e_type
            dw      3                           ;   e_machine
            dd      1                           ;   e_version
            dd      _start                      ;   e_entry
            dd      phdr - $$                   ;   e_phoff
            dd      0                           ;   e_shoff
            dd      0                           ;   e_flags
            dw      ehdrsize                    ;   e_ehsize
            dw      phdrsize                    ;   e_phentsize
            dw      1                           ;   e_phnum
            dw      0                           ;   e_shentsize
            dw      0                           ;   e_shnum
            dw      0                           ;   e_shstrndx

ehdrsize    equ     $ - ehdr

phdr:                                           ; Elf32_Phdr
            dd      1                           ;   p_type
            dd      0                           ;   p_offset
            dd      $$                          ;   p_vaddr
            dd      $$                          ;   p_paddr
            dd      filesize                    ;   p_filesz
            dd      filesize                    ;   p_memsz
            dd      5                           ;   p_flags
            dd      0x1000                      ;   p_align

phdrsize    equ     $ - phdr

_start:

; your program here

filesize    equ     $ - $$
```

在这个模板的帮助下，我们需要做的就是将自己程序的汇编代码写在 _start 下方，是不是 so easy (✪ω✪)！

Step 5:

在完成上述步骤后，我们的可执行文件只剩下了必要的干货，现在可执行文件有多大？还能继续缩小吗？别轻易停下来，后面只剩几页了 ……

- 你遇到过超载吗？当车内已经满员，但司机还想继续塞人时该怎么办呢？简单，后来的人坐在别人腿上就行。（这仅仅只是一个描述——请勿超载，行车不规范，亲人两行泪）——所以，当我们对于不可删减的内容还想压缩时，其中一个方法就是“叠叠乐”或者说强行“缝合”。
- 在 ELF 头中，有一些实际并没作用的内容，例如在标识符末尾那几个滥竽充数的 0 （数一数，共有多少个字节？）。而我们的程序，数一数需要多少个字节即可完成？所以我们是不是可以将程序实现部分放入在标识符末位的填充部分上呢？

```
; tiny.asm

BITS 32

            org     0x08048000

ehdr:                                               ; Elf32_Ehdr
            db      0x7F, "ELF"                     ;   e_ident
            db      1, 1, 1, 0, 0
_start:     mov     bl, 42
            xor     eax, eax
            inc     eax
            int     0x80
            dw      2                               ;   e_type
            dw      3                               ;   e_machine
            dd      1                               ;   e_version
            dd      _start                          ;   e_entry
            dd      phdr - $$                       ;   e_phoff
            dd      0                               ;   e_shoff
            dd      0                               ;   e_flags
            dw      ehdrsize                        ;   e_ehsize
            dw      phdrsize                        ;   e_phentsize
            dw      1                               ;   e_phnum
            dw      0                               ;   e_shentsize
            dw      0                               ;   e_shnum
            dw      0                               ;   e_shstrndx

ehdrsize    equ     $ - ehdr
```

```
phdr:                                                   ; Elf32_Phdr
            dd      1                                   ;   p_type
            dd      0                                   ;   p_offset
            dd      $$                                  ;   p_vaddr
            dd      $$                                  ;   p_paddr
            dd      filesize                            ;   p_filesz
            dd      filesize                            ;   p_memsz
            dd      5                                   ;   p_flags
            dd      0x1000                              ;   p_align

phdrsize    equ     $ - phdr

filesize    equ     $ - $$
```

所以，可不可以继续进行类似尝试，将 Program header 也使用相同的操作置入 ELF header。我们注意到（真是注意力惊人）， ELF header 中的最后 8 个字节与 Program header 中的前 8 个字节具有某种相似之处，而这种相似，可以让我们进一步“缝合”：

```
; tiny.asm

BITS 32

            org     0x08048000

ehdr:
            db      0x7F, "ELF"                 ; e_ident
            db      1, 1, 1, 0, 0
_start:     mov     bl, 42
            xor     eax, eax
            inc     eax
            int     0x80
            dw      2                           ; e_type
            dw      3                           ; e_machine
            dd      1                           ; e_version
            dd      _start                      ; e_entry
            dd      phdr - $$                   ; e_phoff
            dd      0                           ; e_shoff
            dd      0                           ; e_flags
            dw      ehdrsize                    ; e_ehsize
            dw      phdrsize                    ; e_phentsize
phdr:       dd      1                           ; e_phnum       ; p_type
                                                ; e_shentsize
```

```
                dd      0                       ; e_shnum       ; p_offset
                                                ; e_shstrndx
ehdrsize        equ     $ - ehdr
                dd      $$                                      ; p_vaddr
                dd      $$                                      ; p_paddr
                dd      filesize                                ; p_filesz
                dd      filesize                                ; p_memsz
                dd      5                                       ; p_flags
                dd      0x1000                                  ; p_align
phdrsize        equ     $ - phdr

filesize        equ     $ - $$
```

现在我们确实尽可能在缩减了，没有其他部可以再这样“缝合”了，或者你还能找到其他字节？

Step 6:

- 是不是能从结构下手，更改接受的内容呢？请尝试列出所有 ELF header 的各部分内容、作用及其大小！
- 发现了吗？ELF header 中的大部分必要字段都在前半部分，后半部分几乎完全可以自由地进行修改。考虑到这一点，我们可以继续：

```
; tiny.asm

BITS 32

                org     0x00200000

                db      0x7F, "ELF"             ; e_ident
                db      1, 1, 1, 0, 0
_start:
                mov     bl, 42
                xor     eax, eax
                inc     eax
                int     0x80
                dw      2                       ; e_type
                dw      3                       ; e_machine
                dd      1                       ; e_version
                dd      _start                  ; e_entry
                dd      phdr - $$               ; e_phoff
phdr:           dd      1                       ; e_shoff       ; p_type
                dd      0                       ; e_flags       ; p_offset
```

```
                dd      $$                      ; e_ehsize      ; p_vaddr
                                                ; e_phentsize
                dw      1                       ; e_phnum       ; p_paddr
                dw      0                       ; e_shentsize
                dd      filesize                ; e_shnum       ; p_filesz
                                                ; e_shstrndx
                dd      filesize                                ; p_memsz
                dd      5                                       ; p_flags
                dd      0x1000                                  ; p_align

filesize        equ     $ - $$
```

- 此时，程序头表的前 20 个字节与 ELF 头的最后 20 个字节重叠，而且还契合得很好。重叠区域内 ELF 头只有两个部分是重要的：第一个是 e_phnum 字段，它恰好与 p_paddr 字段重合，这是程序头表中为数不多的绝对被忽略的字段之一。另一个是 e_phentsize 字段，它与 p_vaddr 字段的上半部分重合。这些为我们的程序选择一个非标准加载地址来匹配，上半部分等于 0x0020。
- Program header 中还有几个字段可以供我们可以进处理。注意到了嘛，p_memsz 表示要为内存段分配多少内存，即它至少需要与 p_filesz 一样大，但如果它更大，也不会有任何坏处 —— 可以“尸位素餐”。同时我们可以将 p_flags 字段中的可执行位设置为 0，因为可读位和可执行位存在着一种微妙的共生关系（任何一个都会暗示另一个）。基于前述事实，我们将可执行文件重新组织成这个小怪物：

```
; tiny.asm

BITS 32

                org     0x00010000

                db      0x7F, "ELF"             ; e_ident
                dd      1                                       ; p_type
                dd      0                                       ; p_offset
                dd      $$                                      ; p_vaddr
                dw      2                       ; e_type        ; p_paddr
                dw      3                       ; e_machine
                dd      _start                  ; e_version     ; p_filesz
                dd      _start                  ; e_entry       ; p_memsz
                dd      4                       ; e_phoff       ; p_flags
_start:
```

```
                mov     bl, 42                  ; e_shoff       ; p_align
                xor     eax, eax
                inc     eax                     ; e_flags
                int     0x80
                db      0
                dw      0x34                    ; e_ehsize
                dw      0x20                    ; e_phentsize
                dw      1                       ; e_phnum
                dw      0                       ; e_shentsize
                dw      0                       ; e_shnum
                dw      0                       ; e_shstrndx

  filesize      equ     $ - $$
```

- p_flags 字段已从 5（00000101） 更改为 4（00000100），这个 4 也是 e_phoff 字段的值，它给出了程序头表的文件偏移量，这正是我们找到它的位置。我们自己的“程序正文”已下移至 ELF 头的下部，从 e_shoff 字段开始，到 e_flags 字段内结束。

请注意，加载地址已更改为一个低得多的数字 — 实际上，大约是尽可能低的数字。这会将 e_entry 字段中的值保持为一个相当小的数字，这很好，因为它也是 p_memsz 数字（实际上，对于虚拟内存，这几乎无关紧要，为什么？）。

- 对 p_filesz 为什么要如此更改？因为我们没有在 p_flags 字段中设置写入位，所以 Linux 不允许定义大于 p_filesz 的 p_memsz 值，因为如果这些额外的字节不可写，它就无法对它们进行零初始化。由于我们无法在不将程序头表移出对齐的情况下更改 p_flags 字段，因此你可能会认为唯一的解决方案是将 p_memsz 值减小大等于 p_filesz（这将使无法与 e_entry 共享它）。但是，存在另一种解决方案，即将 p_filesz 增加到等于 p_memsz。这意味着它们都比实际文件更大 ，而且要大很多 — 但它使加载程序不必写入只读内存，而这正是它所关心的。
- 现在只剩下最后一个“伎俩”了。即使文件大小不完全符合完整的 ELF 头要求， Linux 系统仍然会 play ball ，自动用零填充缺失的字节，而我们的文件末尾至少有七个零，所以我们可以它们从文件中删除……
- 好啦，现在你的可执行文件是多大？ta 还能如期正常执行吗？